Production Deployment Plan — narofashion.co.tz

> ***HISTORICAL PLAN. DO NOT FOLLOW.*** *This is the original March 2026 plan. Its commands are* ***unsafe on today's shared VPS****:* `pm2 restart all` *restarts other tenants' apps,* `git pull origin master` *pulls the wrong branch,* `prisma db push` *can drop data, and a parallel* `pnpm build` *OOMs the box. Current procedures:* `docs/OPS/DEPLOY.md` *and* `docs/DEPLOYMENT_GUIDE.md` *.*

Context

Deploy the Naro Fashion multi-tenant SaaS platform to production. Domain `narofashion.co.tz` is already secured. The platform consists of 3 apps (storefront, admin, API) + PostgreSQL database, all running on a single VPS behind Nginx with SSL.

Step 1: Choose & Provision a VPS

Recommended: Hetzner Cloud CX32 (~$7.50/mo) or DigitalOcean Droplet ($12/mo)

Spec	Minimum	Recommended
vCPU	2	4
RAM	4 GB	8 GB
Disk	40 GB SSD	80 GB SSD
OS	Ubuntu 24.04 LTS	Ubuntu 24.04 LTS
Location	Europe (Frankfurt)	Europe or Africa-edge

Action: Create the VPS, note the public IP address (e.g., `5.xxx.xxx.xxx`).

Step 2: DNS Configuration

Point these records to your VPS IP at your `.co.tz` registrar:

Type	Name	Value	TTL
A	`@` (narofashion.co.tz)	`<VPS_IP>`	3600
A	`www`	`<VPS_IP>`	3600
A	`admin`	`<VPS_IP>`	3600
A	`api`	`<VPS_IP>`	3600
CNAME	`www`	`narofashion.co.tz`	3600

Result:

- `narofashion.co.tz` → Storefront (port 3000)
- `admin.narofashion.co.tz` → Admin dashboard (port 3001)
- `api.narofashion.co.tz` → NestJS API (port 4000)

Step 3: Server Initial Setup

SSH into the VPS and run:

```
# Update system
sudo apt update && sudo apt upgrade -y

# Install essentials
sudo apt install -y curl git build-essential nginx certbot python3-certbot-nginx ufw

# Install Node.js 22 (LTS)
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs

# Install pnpm
npm install -g pnpm@10

# Install PM2 (process manager)
npm install -g pm2

# Install PostgreSQL 17
sudo apt install -y postgresql-17 postgresql-client-17

# Configure firewall
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'
sudo ufw enable
```

Step 4: PostgreSQL Setup

```
# Switch to postgres user
sudo -u postgres psql

# Inside psql:
CREATE USER naro_admin WITH PASSWORD '<STRONG_PASSWORD_HERE>';
CREATE DATABASE naro_fashion OWNER naro_admin;
GRANT ALL PRIVILEGES ON DATABASE naro_fashion TO naro_admin;
\q
```

Generate a strong password: `openssl rand -base64 32`

Step 5: Clone & Build the Project

```
# Create app directory
sudo mkdir -p /var/www/naro-fashion
sudo chown $USER:$USER /var/www/naro-fashion
```

```
# Clone repo
cd /var/www/naro-fashion
git clone https://github.com/rjurio/naro-fashion.git .

# Install dependencies
pnpm install

# Create production environment file
cp .env.example .env
```

Step 6: Production Environment Variables

Edit `/var/www/naro-fashion/.env`:

```
NODE_ENV=production

# Database (use the strong password from Step 4)
DATABASE_URL="postgresql://naro_admin:<STRONG_PASSWORD>@localhost:5432/naro_fashion?
schema=public"

# JWT Secrets (generate unique ones!)
JWT_SECRET="<run: openssl rand -base64 64>"
JWT_REFRESH_SECRET="<run: openssl rand -base64 64>"
JWT_EXPIRATION="15m"
JWT_REFRESH_EXPIRATION="7d"

# URLs
STOREFRONT_URL="https://narofashion.co.tz"
ADMIN_URL="https://admin.narofashion.co.tz"
API_URL="https://api.narofashion.co.tz"
NEXT_PUBLIC_API_URL="https://api.narofashion.co.tz/api/v1"

# Email (Brevo / SendGrid / any SMTP)
SMTP_HOST="smtp-relay.brevo.com"
SMTP_PORT="587"
SMTP_USER="<your-brevo-login>"
SMTP_PASS="<your-brevo-api-key>"
SMTP_FROM="noreply@narofashion.co.tz"

# SMS (Africa's Talking — optional)
AT_API_KEY="<your-key>"
AT_USERNAME="<your-username>"
AT_SENDER_ID="NaroFashion"

# Instagram Integration
INSTAGRAM_ACCESS_TOKEN="<your-token>"
INSTAGRAM_BUSINESS_ACCOUNT_ID="17841418108905851"
FACEBOOK_APP_ID="4338851449722487"
FACEBOOK_APP_SECRET="<your-secret>"
```

```
# Payment Gateways (configure when ready)
# SELCOM_API_KEY=
# FLUTTERWAVE_SECRET_KEY=
```

Also create frontend env files:

```
# Storefront
echo 'NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1' > apps/storefront/.env.local
echo 'NEXT_PUBLIC_TENANT_SLUG=naro-fashion' >> apps/storefront/.env.local

# Admin
echo 'NEXT_PUBLIC_API_URL=https://api.narofashion.co.tz/api/v1' > apps/admin/.env.local
```

Copy `.env` to database package:

```
cp .env packages/database/.env
```

Step 7: Update Hardcoded localhost References

Two Next.js configs have hardcoded `localhost:4000` in rewrites — must use env var:

`apps/storefront/next.config.js` — change rewrite destination:

```
rewrites: () => [
  { source: '/uploads/:path*', destination:
`${process.env.NEXT_PUBLIC_API_URL?.replace('/api/v1', '') ||
'http://localhost:4000'}/uploads/:path*` }
]
```

`apps/admin/next.config.js` — same change:

```
rewrites: () => [
  { source: '/uploads/:path*', destination:
`${process.env.NEXT_PUBLIC_API_URL?.replace('/api/v1', '') ||
'http://localhost:4000'}/uploads/:path*` }
]
```

Step 8: Build & Migrate Database

```
cd /var/www/naro-fashion

# Generate Prisma client
cd packages/database && npx prisma generate && cd ../..

# Push schema to database
cd packages/database && npx prisma db push && cd ../..
```

```
# Run multi-tenant migration (creates first tenant + platform admin)
node packages/database/prisma/migrate-to-multi-tenant.js

# Seed tenant data (settings, CMS pages, payment methods)
node packages/database/prisma/seed-tenant.js

# Build all apps
pnpm build
```

Step 9: Create PM2 Ecosystem Config

Create `/var/www/naro-fashion/ecosystem.config.js`:

```
module.exports = {
  apps: [
    {
      name: 'naro-api',
      cwd: './apps/api',
      script: 'dist/main.js',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 4000,
      },
    },
    {
      name: 'naro-storefront',
      cwd: './apps/storefront',
      script: 'node_modules/.bin/next',
      args: 'start -p 3000',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 3000,
      },
    },
    {
      name: 'naro-admin',
      cwd: './apps/admin',
      script: 'node_modules/.bin/next',
      args: 'start -p 3001',
      instances: 1,
      env: {
        NODE_ENV: 'production',
        PORT: 3001,
      },
    },
  ],
};
```

Start with PM2:

```
cd /var/www/naro-fashion
pm2 start ecosystem.config.js
pm2 save
pm2 startup  # Follow the printed command to enable auto-start on reboot
```

Step 10: Nginx Configuration

Create `/etc/nginx/sites-available/narofashion`:

```
# Storefront — narofashion.co.tz
server {
    listen 80;
    server_name narofashion.co.tz www.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# Admin — admin.narofashion.co.tz
server {
    listen 80;
    server_name admin.narofashion.co.tz;

    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }
}

# API — api.narofashion.co.tz
server {
    listen 80;
```

```
    server_name api.narofashion.co.tz;
    client_max_body_size 30M;  # For 3D model uploads (25MB)

    location / {
        proxy_pass http://127.0.0.1:4000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
    }

    # Cache static uploads
    location /uploads/ {
        proxy_pass http://127.0.0.1:4000/uploads/;
        proxy_cache_valid 200 7d;
        add_header Cache-Control "public, max-age=604800";
    }
}
```

Enable and start:

```
sudo ln -s /etc/nginx/sites-available/narofashion /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t  # Test config
sudo systemctl reload nginx
```

Step 11: SSL Certificates (Let's Encrypt)

```
sudo certbot --nginx -d narofashion.co.tz -d www.narofashion.co.tz -d
admin.narofashion.co.tz -d api.narofashion.co.tz --email hello@narofashion.co.tz --agree-tos
--non-interactive
```

Certbot auto-configures Nginx for HTTPS and sets up auto-renewal. Verify:

```
sudo certbot renew --dry-run
```

Step 12: Update Tenant Domain in Database

After deployment, set the production domain for the Naro Fashion tenant:

```
cd /var/www/naro-fashion
node -e "
  const { PrismaClient } = require('@prisma/client');
```

```
  const prisma = new PrismaClient();
  prisma.tenant.update({
    where: { slug: 'naro-fashion' },
    data: { domain: 'narofashion.co.tz' }
  }).then(t => { console.log('Domain set:', t.domain); prisma.\$disconnect(); });
"
```

Step 13: Create Deployment Script

Create `/var/www/naro-fashion/deploy.sh`:

```
#!/bin/bash
set -e

echo "🚀 Deploying Naro Fashion..."

cd /var/www/naro-fashion

# Pull latest code
git pull origin master

# Install dependencies
pnpm install

# Generate Prisma client & push schema
cd packages/database
npx prisma generate
npx prisma db push --accept-data-loss
cd ../..

# Build all apps
pnpm build

# Restart PM2 processes
pm2 restart ecosystem.config.js

echo "✅ Deployment complete!"
```

Make executable: `chmod +x deploy.sh`

Usage: `./deploy.sh`

Step 14: GitHub Actions CI/CD (Optional)

Create `.github/workflows/deploy.yml`:

```
name: Deploy to Production

on:
  push:
```

```
    branches: [master]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ secrets.VPS_HOST }}
          username: ${{ secrets.VPS_USER }}
          key: ${{ secrets.VPS_SSH_KEY }}
          script: |
            cd /var/www/naro-fashion
            ./deploy.sh
```

GitHub Secrets to set:

- `VPS_HOST` — Your VPS IP
- `VPS_USER` — SSH user (e.g., `deploy`)
- `VPS_SSH_KEY` — Private SSH key

Step 15: Post-Deployment Checklist

#	Task	Command/Action
1	Verify storefront loads	`curl -I https://narofashion.co.tz`
2	Verify admin loads	`curl -I https://admin.narofashion.co.tz`
3	Verify API responds	`curl https://api.narofashion.co.tz/api/v1/tenants/resolve?slug=naro-fashion`
4	Test admin login	Login at `admin.narofashion.co.tz` with `admin@narofashion.co.tz` / `admin123`
5	Test platform login	Login at `admin.narofashion.co.tz/platform-login` with `platform@naro.co.tz` / `Admin123`
6	**Change default passwords**	Update admin + platform admin passwords immediately
7	Test product images	Browse products, verify images load from `/uploads/`
8	Test mobile	Access `narofashion.co.tz` on phone
9	Test SSL	Verify padlock icon, check `https://` works
10	Set up monitoring	`pm2 install pm2-logrotate` for log management
11	Set up backups	Cron: `pg_dump naro_fashion > /backups/naro_$(date +%F).sql` daily
12	Configure email	Test order confirmation email flow

Step 16: Ongoing Maintenance

Database Backups (daily cron)

```
# Add to crontab (crontab -e):
0 2 * * * pg_dump -U naro_admin naro_fashion | gzip > /var/backups/naro_fashion_$(date
+\%F).sql.gz
0 3 * * * find /var/backups -name "naro_fashion_*.sql.gz" -mtime +30 -delete
```

Log Management

```
pm2 install pm2-logrotate
pm2 set pm2-logrotate:max_size 10M
pm2 set pm2-logrotate:retain 7
```

Monitor

```
pm2 monit          # Live monitoring
pm2 logs           # View all logs
pm2 logs naro-api  # View API logs only
```

Update Deployment

```
cd /var/www/naro-fashion && ./deploy.sh
```

Cost Summary (Monthly)

Service	Cost
VPS (Hetzner CX32)	~$7.50/mo
Domain (.co.tz)	~$2.50/mo ($30/yr)
SSL (Let's Encrypt)	Free
Email (Brevo free tier)	Free (300 emails/day)
SMS (Africa's Talking)	Pay-per-use (~$0.02/SMS)
Total	**~$10/mo**

Security Hardening

1. **SSH:** Disable password auth, use SSH keys only
2. **Firewall:** Only ports 22, 80, 443 open
3. **PostgreSQL:** Listen only on localhost (default)
4. **JWT secrets:** Unique, 64+ bytes, never committed to git
5. **Admin passwords:** Change defaults immediately after deploy
6. **Rate limiting:** Already configured (100 req/60s via NestJS Throttler)

7. **Helmet:** Already enabled (CSP, XSS, clickjack protection)
8. **File uploads:** Size limits enforced (5MB images, 25MB 3D models)